

```
!nvidia-smi
```

```
Tue Apr 28 05:28:57 2026
```

```
+-----+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----+-----+-----+-----+-----+-----+
| GPU   Name                               Persistence-M   Bus-Id        Disp.A   Volatile Uncorr. ECC   |
| Fan  Temp  Perf              Pwr:Usage/Cap     Memory-Usage   GPU-Util    Compute M.   |
|                                           MIG M.       |
+-----+-----+-----+-----+-----+-----+
|   0   Tesla T4                        Off               00000000:00:04.0 Off   |
| N/A   40C    P8              9W / 70W         0MiB / 15360MiB      0%          Default |
+-----+-----+-----+-----+-----+-----+
|                                           N/A          |
+-----+-----+-----+-----+-----+

```

```
+-----+
| Processes:
| GPU   GI    CI          PID    Type    Process name                        GPU Memory
|      ID    ID                                   Usage
+-----+-----+-----+-----+-----+
| No running processes found
+-----+

```

Start coding or [generate](#) with AI.

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import kagglehub
import os

# Download dataset
path = kagglehub.dataset_download("paultimothymooney/chest-xray-pneumonia")
train_dir = os.path.join(path, 'chest_xray', 'train')
test_dir = os.path.join(path, 'chest_xray', 'test')

# Setup Generators (Simplified for speed)
IMG_SIZE = (224, 224)
BATCH_SIZE = 32

train_datagen = ImageDataGenerator(rescale=1./255) # Removed augmentation for speed
test_datagen = ImageDataGenerator(rescale=1./255)

train_gen = train_datagen.flow_from_directory(train_dir, target_size=IMG_SIZE, batch_size=BATCH_SIZE, class_mode='binary')
test_gen = test_datagen.flow_from_directory(test_dir, target_size=IMG_SIZE, batch_size=BATCH_SIZE, class_mode='binary')

def fast_train(model, name):
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    print(f"--- Fast Training {name} ---")
    # Training for 3 epochs with limited validation steps to save time
    model.fit(train_gen, epochs=3, validation_data=test_gen, validation_steps=10)
    tf.keras.backend.clear_session() # Frees RAM immediately
```

Using Colab cache for faster access to the 'chest-xray-pneumonia' dataset.
Found 5216 images belonging to 2 classes.
Found 624 images belonging to 2 classes.

```
cnn = models.Sequential([
    layers.Conv2D(16, (3, 3), activation='relu', input_shape=(224, 224, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
fast_train(cnn, "CNN")
```

```
--- Fast Training CNN ---
```

```
Epoch 1/3
```

```
163/163 ————— 65s 383ms/step - accuracy: 0.8903 - loss: 0.7699 - val_accuracy: 0.6938 -
```

```
Epoch 2/3
```

```
163/163 ————— 82s 500ms/step - accuracy: 0.9686 - loss: 0.0865 - val_accuracy: 0.7656 -
```

```
Epoch 3/3
```

```
163/163 ————— 52s 318ms/step - accuracy: 0.9780 - loss: 0.0613 - val_accuracy: 0.8156 -
```

```
alexnet = models.Sequential([
    layers.Conv2D(48, 11, strides=4, activation='relu', input_shape=(224,224,3)),
    layers.MaxPooling2D(3, strides=2),
    layers.Flatten(),
    layers.Dense(512, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
fast_train(alexnet, "AlexNet")
```

```
--- Fast Training AlexNet ---
```

```
Epoch 1/3
```

```
163/163 ————— 55s 321ms/step - accuracy: 0.8719 - loss: 0.3445 - val_accuracy: 0.7781 -
```

```
Epoch 2/3
```

```
163/163 ————— 52s 319ms/step - accuracy: 0.9503 - loss: 0.1290 - val_accuracy: 0.7812 -
```

```
Epoch 3/3
```

```
163/163 ————— 51s 313ms/step - accuracy: 0.9636 - loss: 0.0916 - val_accuracy: 0.7312 -
```

```
vgg16_base = tf.keras.applications.VGG16(weights='imagenet', include_top=False, input_shape=(224,224,3))
vgg16_base.trainable = False
vgg16 = models.Sequential([vgg16_base, layers.GlobalAveragePooling2D(), layers.Dense(1, activation='sigmoid')])
fast_train(vgg16, "VGG16")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16\_weights\_58889256/58889256 ————— 0s 0us/step
```

```
--- Fast Training VGG16 ---
```

```
Epoch 1/3
```

```
163/163 ————— 80s 384ms/step - accuracy: 0.7354 - loss: 0.5234 - val_accuracy: 0.6000 -
```

```
Epoch 2/3
```

```
163/163 ————— 63s 384ms/step - accuracy: 0.7872 - loss: 0.4017 - val_accuracy: 0.6719 -
```

```
Epoch 3/3
```

```
163/163 ————— 63s 388ms/step - accuracy: 0.8591 - loss: 0.3360 - val_accuracy: 0.7063 -
```

```
vgg19_base = tf.keras.applications.VGG19(weights='imagenet', include_top=False, input_shape=(224,224,3))
vgg19_base.trainable = False
vgg19 = models.Sequential([vgg19_base, layers.GlobalAveragePooling2D(), layers.Dense(1, activation='sigmoid')])
fast_train(vgg19, "VGG19")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg19/vgg19\_weights\_80134624/80134624 ————— 1s 0us/step
```

```
--- Fast Training VGG19 ---
```

```
Epoch 1/3
```

```
163/163 ————— 70s 413ms/step - accuracy: 0.7301 - loss: 0.5223 - val_accuracy: 0.6438 -
```

```
Epoch 2/3
```

```
163/163 ————— 82s 501ms/step - accuracy: 0.7916 - loss: 0.4080 - val_accuracy: 0.6594 -
```

```
Epoch 3/3
```

```
163/163 ————— 68s 413ms/step - accuracy: 0.8516 - loss: 0.3496 - val_accuracy: 0.7281 -
```

```
m_base = tf.keras.applications.MobileNet(weights='imagenet', include_top=False, input_shape=(224,224,3))
m_base.trainable = False
mobilenet = models.Sequential([m_base, layers.GlobalAveragePooling2D(), layers.Dense(1, activation='sigmoid')])
fast_train(mobilenet, "MobileNet")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet/mobilenet\_17225924/17225924 ————— 0s 0us/step
```

```
--- Fast Training MobileNet ---
```

```
Epoch 1/3
```

```
163/163 ————— 71s 354ms/step - accuracy: 0.9245 - loss: 0.2063 - val_accuracy: 0.8281 -  
Epoch 2/3  
163/163 ————— 55s 337ms/step - accuracy: 0.9620 - loss: 0.1078 - val_accuracy: 0.7969 -  
Epoch 3/3  
163/163 ————— 53s 322ms/step - accuracy: 0.9674 - loss: 0.0890 - val_accuracy: 0.8250 -
```

```
x_base = tf.keras.applications.Xception(weights='imagenet', include_top=False, input_shape=(224,224,3))  
x_base.trainable = False  
xception = models.Sequential([x_base, layers.GlobalAveragePooling2D(), layers.Dense(1, activation='sigmoid')])  
fast_train(xception, "Xception")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/xception/xception\_weights\_tf\_dim\_ordering\_tf\_data\_format.h5  
83683744/83683744 ————— 0s 0us/step  
--- Fast Training Xception ---  
Epoch 1/3  
163/163 ————— 88s 398ms/step - accuracy: 0.8938 - loss: 0.2677 - val_accuracy: 0.7844 -  
Epoch 2/3  
163/163 ————— 62s 377ms/step - accuracy: 0.9415 - loss: 0.1533 - val_accuracy: 0.7937 -  
Epoch 3/3  
163/163 ————— 61s 371ms/step - accuracy: 0.9523 - loss: 0.1284 - val_accuracy: 0.7406 -
```